

Masters' Project Report

**Title: Modeling Path Dependent Financial Products
using Monte Carlo approach**

By: Aarsh Sachdeva

A#: A20595501

Table of Contents

Abstract.....	2
What is Monte Carlo?	2
Financial Concepts	2
Need for Monte Carlo in ACBRC product	3
How do we generate price paths?	4
Payout Computation	5
Vanilla Monte Carlo (VMC).....	6
Antithetic Variates (AV)	6
Control Variates (CV).....	7
RESULTS (VMC v/s AV v/s CV)	8
Experiment 1	9
Experiment 2	10
Pseudo-Random Numbers	12
Sobol Sequence Method	13
RESULTS (VMC v/s Sobol)	15
Conclusions	17
References	18

Abstract

This report investigates the pricing of Auto Call Barrier Reverse Convertibles (ACBRCs) complex path-dependent structured financial products whose multi-asset nature, continuous barrier monitoring, and discontinuous payoff structure render closed-form pricing solutions infeasible, necessitating simulation-based approaches. Four Monte Carlo techniques are implemented and evaluated — Vanilla Monte Carlo (VMC), Antithetic Variates (AV), Control Variates (CV), and Sobol Quasi-Monte Carlo — with performance compared across mean payout accuracy, standard error, computation time, and cost efficiency measured by $SE^2 \times \text{Time}$. To simulate correlated multi-asset price paths, Cholesky decomposition is applied to preserve the observed correlation structure between underlying assets. Furthermore, dimensionality reduction techniques including Principal Component Analysis (PCA) and Cholesky-PCA are incorporated within the Sobol framework to align the highest-variance dimensions with the leading Sobol dimensions, enhancing the effectiveness of quasi-random sampling in high-dimensional settings. The findings provide practical recommendations for the most efficient simulation approach for pricing complex structured products such as ACBRCs.

What is Monte Carlo?

Monte Carlo simulation is a computational method that uses repeated random sampling to model uncertainty and estimate the distribution of possible outcomes in a system.

It is especially useful in finance because markets are inherently uncertain - Monte Carlo helps simulate asset prices, portfolio returns, and risk (e.g., Value at Risk) under many scenarios, rather than relying on a single prediction. This makes it powerful for risk management, option pricing, and decision-making under uncertainty.

Financial Concepts

Options are fundamental derivative contracts where the payoff depends on the price of a single underlying asset at maturity. A call gives the right to buy, and a put gives the right to sell, making them the building blocks of financial engineering because they allow investors to express views on direction, volatility, and risk management in a controlled payoff structure.

Basket Options generalize this idea by linking the payoff to a weighted combination of multiple underlying assets instead of just one. This introduces diversification and correlation effects, meaning the final payoff depends not only on individual asset movements but also on how those assets move together, which is crucial in multi-asset risk exposure and portfolio-based derivatives.

Path-dependent financial products take it further by making the payoff depend on the entire price trajectory of the underlying, not just the final price. Examples include barrier options and Asian options, where features like knock-in/knock-out levels or averaging mechanisms introduce memory into the contract, making pricing more complex and typically requiring simulation methods like Monte Carlo.

Auto call Barrier Reverse Convertibles (ACBRCs) are structured products built on these path-dependent and option-like features, combining an auto call trigger (early redemption if conditions are met) with a barrier reverse convertible structure that offers high coupons in exchange for downside risk. Their payoff depends on whether the underlying stays above or breaches certain levels over time, effectively embedding options within a yield-generating note issued OTC by banks.

Autocall: Autocall is checked at discrete observation dates, and if **all** the underlying assets are above the autocall level which is often S_0 (the product terminates early and pays principal + accrued coupons).

$$S_t \geq S_0 \text{ @ checkpoints} \Rightarrow \textit{Autocall}$$

Barrier: The check if **any** underlying asset reaches the barrier anytime during the lifetime of the product which means that the monitoring is continuous (Monitoring is performed by checking price at time steps and between time steps using **Brownian Bridge formula**). Barrier breach results in the payout according to the terminal price + coupons accrued till the terminal.

$$S_t \geq x.S_0 \text{ @ anytime} \Rightarrow \textit{Barrier Breach}$$

Occurrence of autocall post a barrier breach overturns it.

Need for Monte Carlo in pricing ACBRC product

Path independent options like European options are relatively simple to understand and price because their payoff depends only on the final price of a single underlying asset at maturity, regardless of how the price moved along the way. This path-independence is what makes them mathematically tractable and allows us to derive closed-form pricing solutions like the Black-Scholes formula under standard assumptions, giving a clean and direct relationship between inputs (volatility, interest rate, strike) and the option price.

In contrast, ACBRCs (Auto call Barrier Reverse Convertibles) being path dependent, especially basket-based ones, are far more complex because they depend on multiple underlying assets and the entire path these assets take over time. Their payoff is shaped by several interacting features such as correlation between assets, barrier levels that can trigger losses, and auto call conditions that may lead to early redemption. This combination of path dependency, multi-asset dynamics, and discontinuous payoff rules removes the possibility of a closed-form solution. As a result, unlike European options, ACBRCs cannot be priced neatly using a single equation and instead require simulation-based approaches like Monte Carlo to capture the wide range of possible market outcomes.

Monte Carlo methods are well-suited for pricing ACBRCs because they handle complexity that analytical formulas cannot capture. By simulating many possible future price paths, they naturally incorporate multi-asset dynamics, correlations, and path-dependent features such as barriers and auto call triggers. This makes them particularly effective for products where the payoff depends on the entire evolution of market variables rather than just the final value.

Parameters used during the project:

Number of stocks (n) = 3

Investment (IV) = SG\$ 1000

Dividend (d) = [0.0444, 0.0617, 0.0293] (% annually)

Initial Price (S_0) = [218.75, 23.7, 157.1]

Contract Tenure (T_N) = 1.25 yrs

Volatility (σ): [0.00780911, 0.008202997, 0.007596542]

Correlation between assets = [[1, 0.874028799, 0.798699928], [0.874028799, 1, 0.759168398], [0.798699928, 0.759168398, 1]]

Coupons = 0.08*IV (i.e. 8% Annually)

Autocall checkpoints = [0.5, 0.75, 1] (in yrs)

Number of time steps (t) = 100

Days in trading year = 252

How Do We Generate Price Paths?

To price path-dependent products like ACBRCs, we first need to simulate how the underlying asset evolves over time. This is done by modelling the asset price as a stochastic process and then generating many possible future scenarios using small time-step simulations.

Stochastic Process Assumption (Geometric Brownian Motion): We assume the underlying asset follows a Geometric Brownian Motion, where price movements are driven by two components: a deterministic drift (representing expected return) and a random shock/ diffusion term (representing market uncertainty). This framework is widely used in finance as it captures both trend and volatility in a consistent way.

Discretization of Time: Since continuous price evolution cannot be directly simulated, the time horizon is divided into small intervals. At each step, the asset price is updated sequentially based on its previous value, allowing us to approximate the continuous process in a practical way.

Introducing Randomness: At every time step, randomness is introduced using normally distributed random variables. These shocks capture unpredictable market movements and ensure that each simulated path behaves differently, reflecting real-world uncertainty. Since ACBRCs depend on multiple underlying assets, the joint evolution of asset prices must account for the correlation structure between them. To introduce this correlation into the simulation, the Cholesky decomposition of the correlation matrix is applied.

Given a correlation matrix Σ , the Cholesky decomposition factorizes it as:

$$\Sigma = LL^T$$

where L is a lower triangular matrix. Independent standard normal random variables Z are then transformed as:

$$\mathbf{Z}_{corr} = \mathbf{L} \cdot \mathbf{Z}$$

This ensures that the simulated asset paths exhibit the same pairwise correlations as observed in the market, preserving the dependence structure between assets throughout the simulation.

Generation of Multiple Price Paths: By repeating this step-by-step evolution many times, we generate a large set of possible future price trajectories. Each path represents one potential market scenario, and together they form the foundation for evaluating payoffs in Monte Carlo-based pricing of structured products.

$$\text{GBM: } S_{t+dt} = S_t \cdot \exp((r - d - 0.5 * \sigma^2)dt) + S_t \cdot \exp(\sigma * \sqrt{dt} * Z)$$

$$\text{Brownian Bridge: } S_{\min(t \rightarrow t+1)} = \exp\left(\frac{\ln S_t + \ln S_{t+1}}{2} - \sqrt{\left(\frac{\ln S_t - \ln S_{t+1}}{2}\right)^2 - 0.5 * \sigma^2 * dt * \ln U}\right)$$

S_t : Price at time t

r : Risk free return rate

d : dividend

σ^2 : Annual Volatility

Z : Normal random variable tensor ($Z \sim N(0,1)$)

U : Uniform random variable tensor ($U \sim U[0,1]$)

Average Payout Computation

- Once we have our Z & U generated, we would simulate the paths and minimum price between 2 time points from them respectively.
- Steps to determine the payout for every simulation:
 - Check 1: We will check at each autocall checkpoint if the price exceeds S_0 (autocall threshold).
Payout = $IV * \exp(-r * T_{auto}) + \text{Coupons accrued till } T_{auto}$
 - Check 2: After it we would check remaining simulations if at any time stamp or between time stamp the price goes below barrier ($0.59 * S_0$ in this case).

$$\text{Payout} = IV * \exp(-r * T_N) * S_{T_N} + \text{Coupons}_{all} \text{ where } S_{T_N} = \begin{cases} S_{T_N}, & S_{T_N} < S_0 \\ 1, & \text{otherwise} \end{cases}$$

3. Check 3: If the simulation did not have an autocall reached or barrier breached.

$$\text{Payout} = IV \cdot \exp(-r \cdot T_N) + \text{Coupons}_{all}$$

Getting started with Monte Carlo computation

The project uses the following Monte Carlo techniques to model the price of the product:

- Vanilla Monte Carlo
- Antithetic Variate
- Control Variate
- Sobol Sampling

Vanilla Monte Carlo (VMC)

Vanilla Monte Carlo is the standard simulation-based approach used for pricing financial derivatives. It estimates the value of a product by generating many possible future price paths for the underlying asset and calculating the payoff for each scenario.

The simulated payoffs are then averaged and discounted to present value to obtain the estimated price.

Despite its simplicity, the method is highly flexible and well-suited for complex and path-dependent products such as ACBRCs.

Steps to compute average payout using VMC:

- Simulate N for the underlying asset using Z.
- Compute the payoff of the financial product for each simulated path.
- Discount all simulated payoffs back to present value using the risk-free rate.
- Estimate the product price by taking the average of the discounted payoffs.
- Compute the Mean, Variance, and Standard Error of the estimator.

Antithetic Variates (AV)

Antithetic Variates is a variance reduction technique used to improve the efficiency of Monte Carlo simulation. Instead of generating completely independent random paths, each random shock is paired with its negative counterpart, creating two complementary simulation paths.

Antithetic Variates is one of the simplest and most effective variance reduction techniques. It basically forces symmetry in randomness to cancel noise.

Since the positive and negative shocks tend to offset each other, the variability of the estimator is reduced, leading to more stable pricing results and faster convergence without increasing computational complexity significantly.

In this technique we use negatively correlated random variables of the current random variables.

Instead of simulating a Z or U with N simulations (Used in Vanilla Monte Carlo), we use $N/2$ simulations and simulate Z and U .

The combination of random variable gives N number of simulations and performance comparison of Antithetic Variates with Vanilla Monte Carlo becomes feasible.

$$\text{Payout for AV, } Payout_{AV} = \frac{Payout_+ + Payout_-}{2}$$

If the selected random variables are adequately negatively correlated, we observe that $E[Payout_{AV}] \approx E[Payout_{VMC}]$ and $Var[Payout_{AV}] < Var[Payout_{VMC}]$

Steps to compute average payout using AV:

- Generate a set of Z & U for the simulation of asset price paths and Brownian Bridge.
- Construct corresponding antithetic shocks $-Z$ and $1-U$ to create paired simulation paths.
- Compute the payoff for both the original and antithetic paths.
- Average the paired payoffs to obtain the antithetic estimator with reduced variance.
- Compute the Mean, Variance, and Standard Error of the estimator.

Control Variates (CV)

Control Variates is a variance reduction method that improves Monte Carlo estimates by using a related product with a known analytical value as a reference. The difference between the simulated and known value of the control product is used to adjust the estimator.

By exploiting the correlation between the target payoff and the control variable, the method reduces simulation error and improves pricing accuracy. This makes it particularly effective when a strongly correlated benchmark product is available.

In most cases it performs better than VMC given that certain requirements and assumptions are met:

- 1) We need the control variable X to have high correlation with target variable.
- 2) $E[X]$ should be known.
- 3) $E(X - E[X]) = 0$
- 4) The control variate should have finite variance
- 5) If we have more than one control variable these should not be highly collinear within themselves. In the ideal case scenario, they should exhibit least correlation possible.

$$\text{CV equation, } \mu_{CV} = \mu_{VMC} - \beta(X - E[X])$$

μ_{CV} : CV estimator (CV based payout)

μ_{VMC} : Original estimator (VMC based payout)

X : Control Variate Vector

β : CV Coefficient

Optimal $\beta, \beta^* = \Sigma_{XX}^{-1} \Sigma_{XY}$

Σ_{XX} : Cov matrix of X

Σ_{XY} : Cov column matrix of the μ_{VMC} and X

Steps to compute average payout using CV:

- Simulate the target payoff variable Y , whose variance is to be reduced using a control variate technique. Identify a related control variable X with a known expected value $E[X]$, and compute both X and Y from the same simulations.
- Center the variables by constructing $X' = X - E[X]$ and $Y' = Y - \bar{Y}$.
- Perform a linear regression of Y on X to estimate the optimal coefficient β , and construct the adjusted estimator $Y^{CV} = Y - \beta(X - E[X])$.
- Compute the Mean, Variance, and Standard Error of the variance-reduced estimator Y^{CV} .

Here CV used were:

- 1) Discounted mean terminal payout

$$X = S_T * e^{-rT} \quad \& \quad E[X] = S_0 * e^{-d}$$

- 2) European Put option

$$X = \text{Max}(S_T - K, 0) \quad \& \quad E[X] = K e^{-r} \phi(-d_2) - S_0 \phi(-d_1) \quad (* \text{Black Scholes estimate for put options})$$

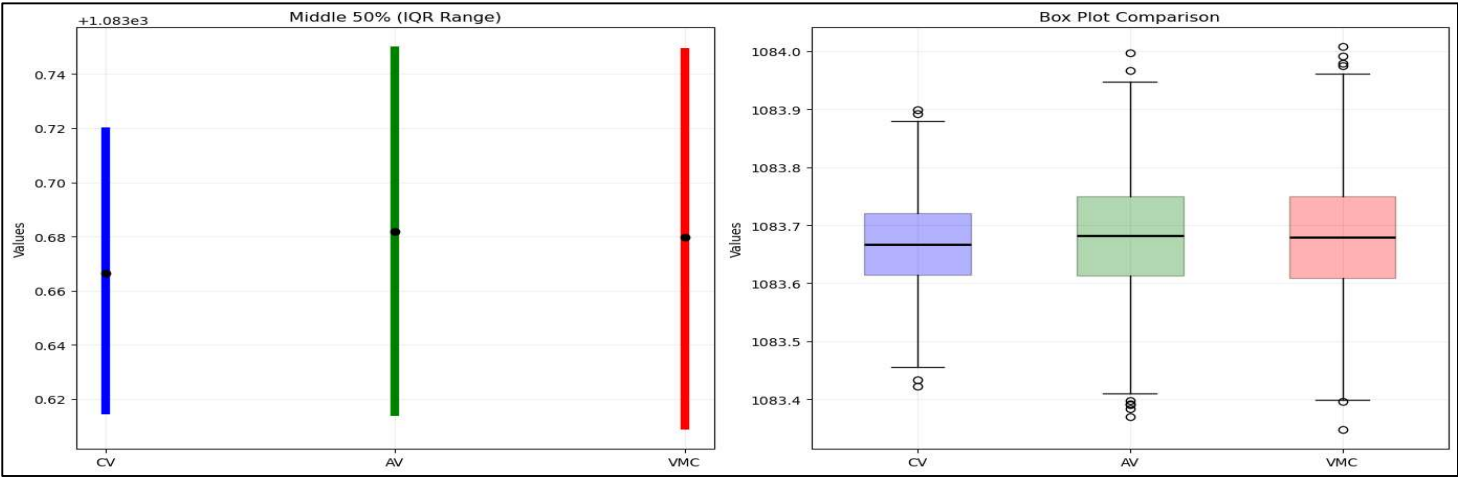
$$\text{where } d_1 = \frac{\log(\frac{S_0}{K}) + (r - d + 0.5\sigma^2)T}{T\sqrt{\sigma}}$$

$$d_2 = d_1 - T\sqrt{\sigma}$$

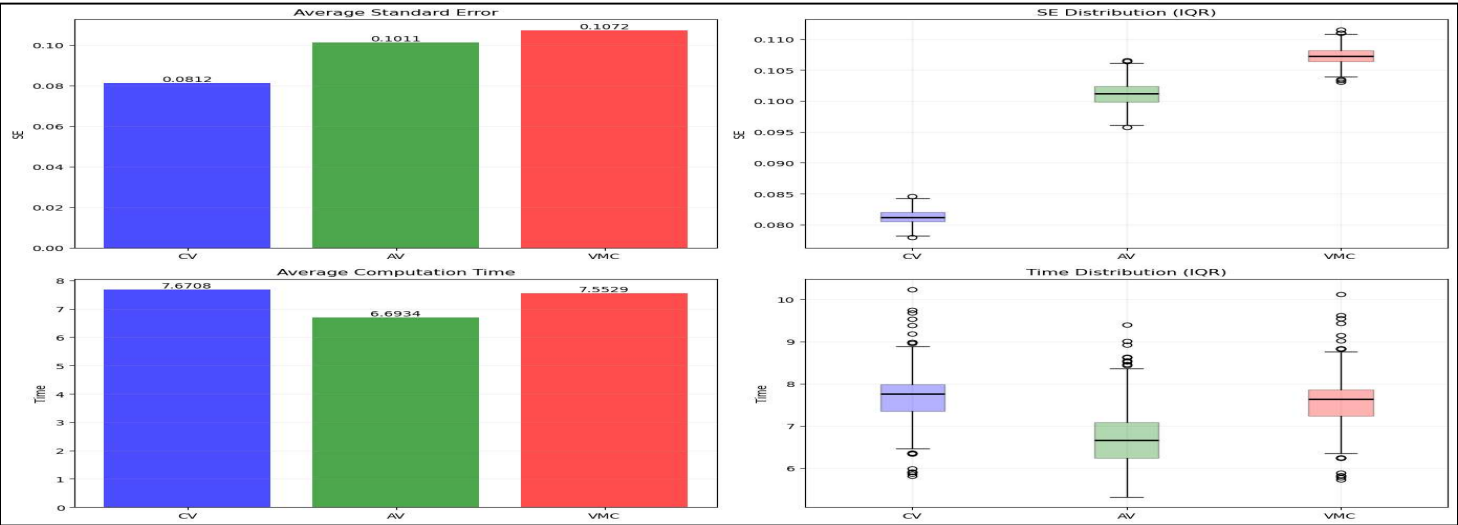
RESULTS:

- To determine and compare the performances of VMC, CV and AV 2 experiments were carried out:
- 1) 100K simulations for 100 iterations
 - 2) N simulations for 100 iterations were $N \in [10^0, 10^1, \dots, 10^5]$

EXPERIMENT 1



(Figure 1)

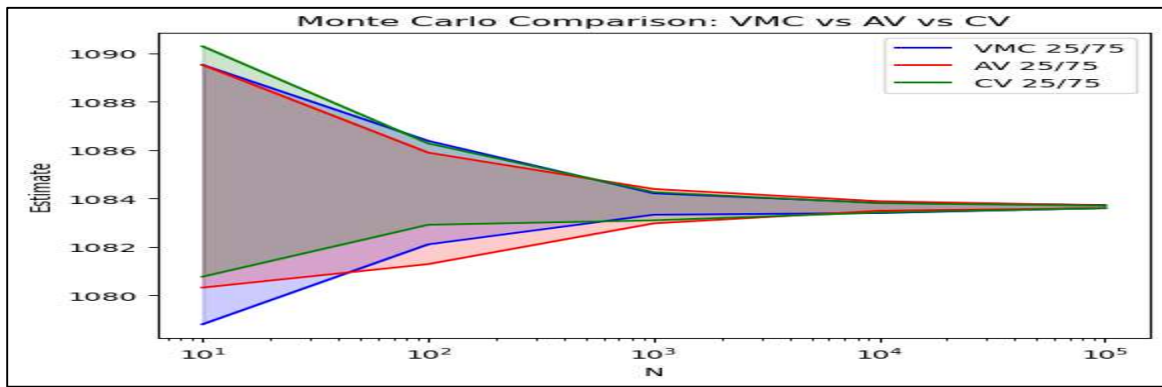


(Figure 2)

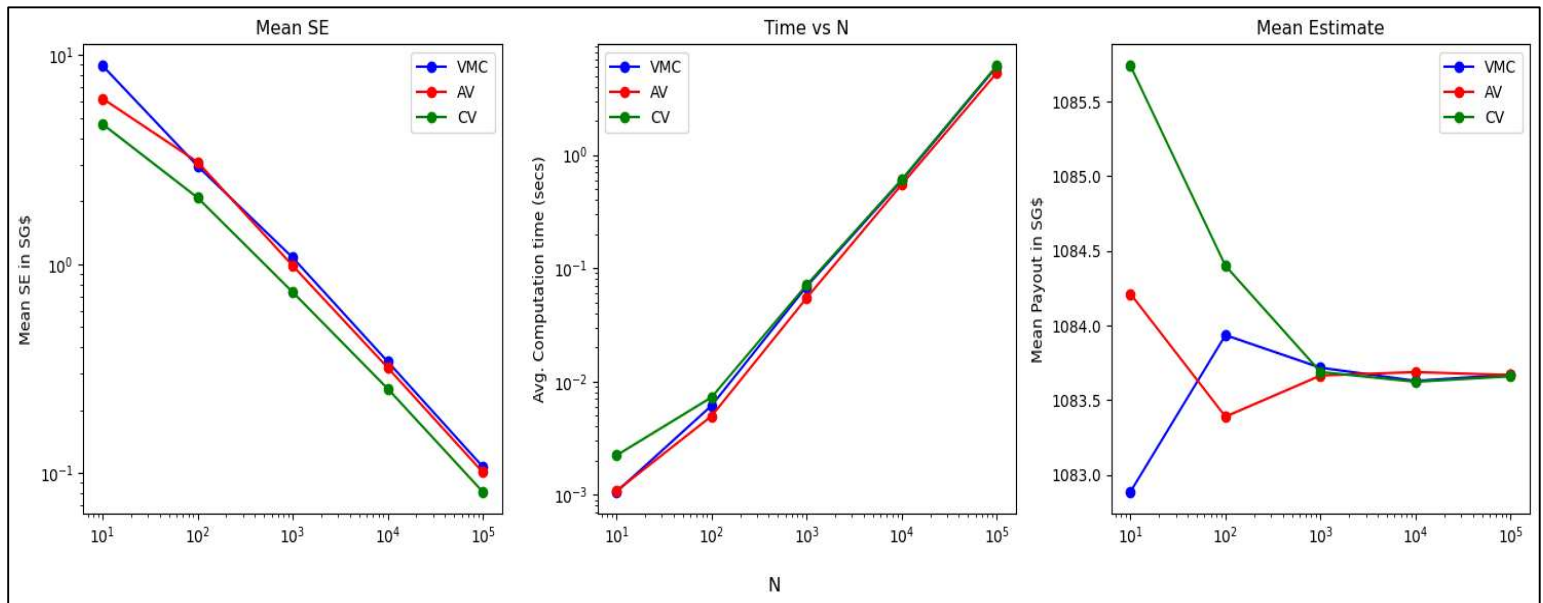
Findings:

- AV has variance similar to VMC as the ACBRC as a product is disjointed and may not produce an ideal symmetry for near barrier and near autocall paths which affect the performance while one advantage over the VMC technique can be the reduced time of computation as N increases.
- When the number of simulations is large the VMC technique outperforms CV and AV considerably in terms of variance reduction. While the computation time is slightly greater than VMC due to the addition arithmetic & matrix operations like centering, optimal β computation, Y_{CV} computation etc.

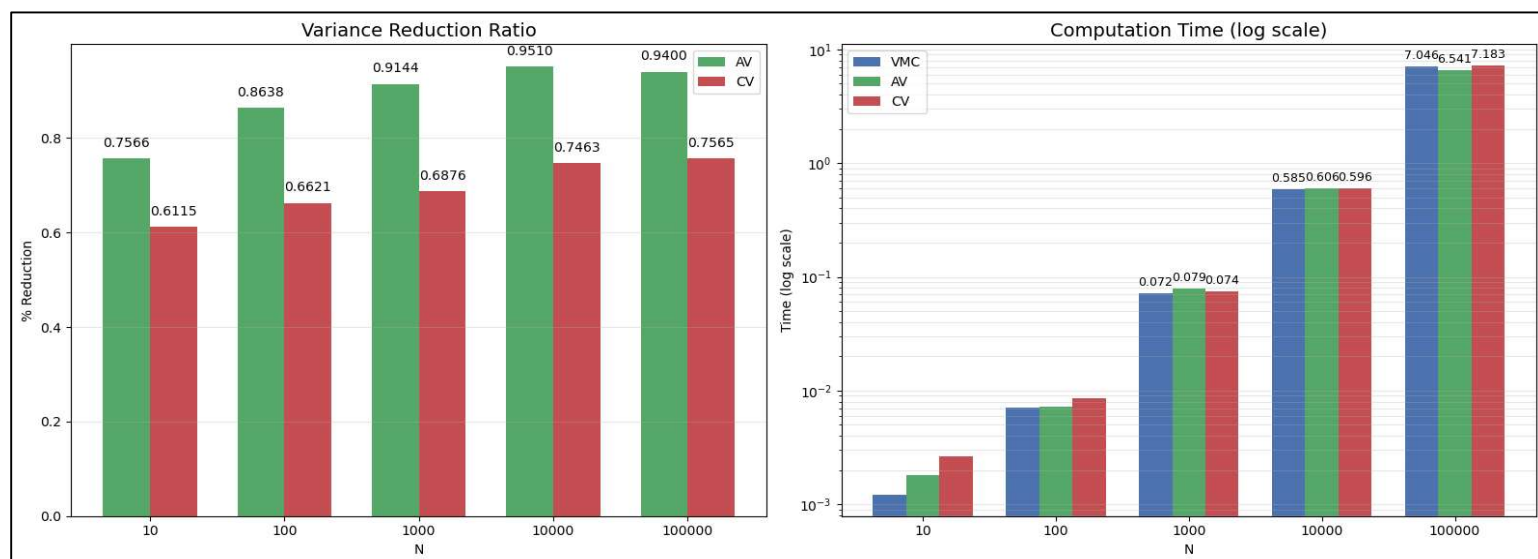
EXPERIMENT 2



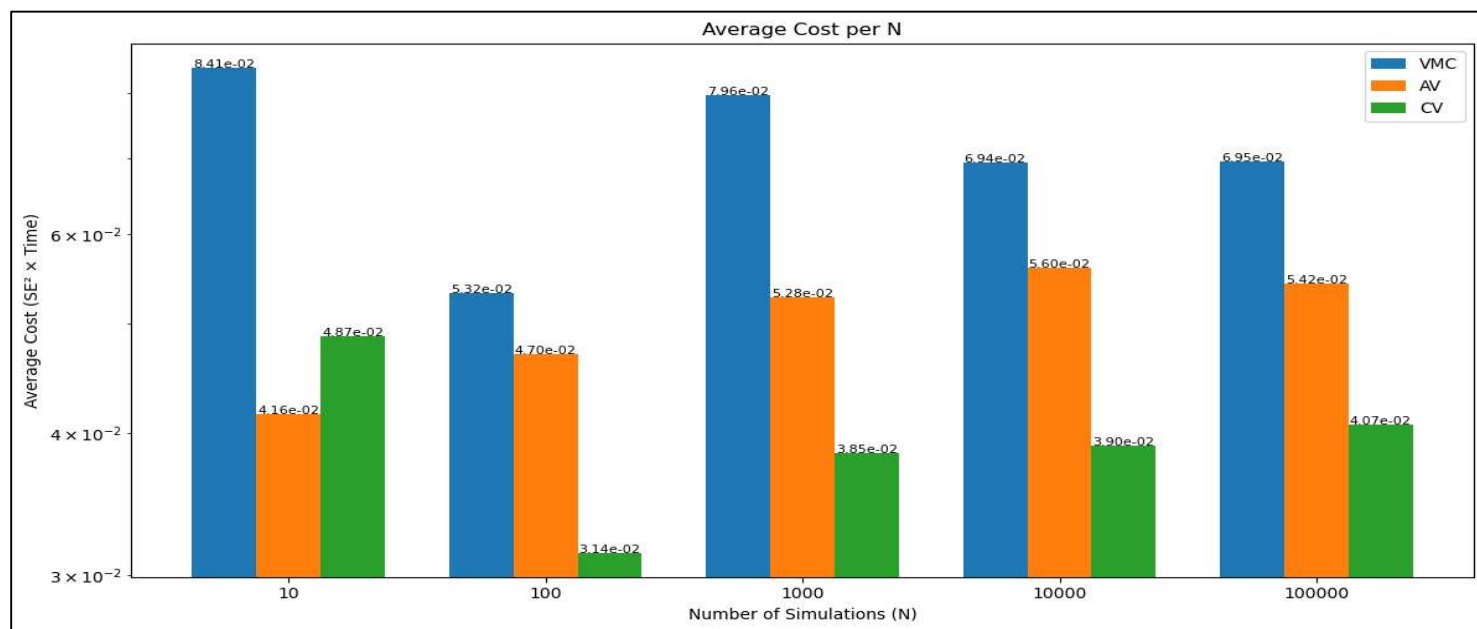
(Figure 3)



(Figure 4)



(Figure 5)



(Figure 6)

Findings:

- The interquartile range (IQR) in figure 3, representing the middle 50% of outcomes, shows higher variability for small N , but becomes increasingly stable across all three techniques as

N increases. This behavior is explained by the Law of Large Numbers, which ensures convergence of the sample mean payout to the true expected value.

- Upon plotting the standard error (SE) against N on a log-log scale in figure 4, the SE decreases linearly, demonstrating the inverse square-root relationship $SE \propto \frac{1}{\sqrt{N}}$, which is consistent with Monte Carlo convergence theory.
- Upon plotting time taken against N on a log-log scale in figure 5, the time increases linearly, indicating a power-law relationship $\text{Time} \propto N$. This is consistent with Monte Carlo simulation complexity, where computational cost grows linearly with the number of simulations.
- The estimated mean payout shows high variability for small sample sizes across the 3 techniques, but as N increases, all estimates converge and stabilize near the true expected value, consistent with the Law of Large Numbers.
- Across different values of N in figure 5, there is a reduction in standard error (SE) of approximately 15–20% for Antithetic Variates (AV) and 30–40% for Control Variates (CV) compared to VMC, demonstrating the effectiveness of variance reduction techniques.
- On plotting computation time against N in figure 5, AV initially require slightly more time due to the additional arithmetic operations involved in computing paired samples and the mean, which can be more expensive than generating the extra $\frac{N}{2}$ paths. However, as N increases, AV becomes the most computationally efficient among the methods. In contrast, CV remain computationally more expensive than both VMC and AV due to additional post-processing steps required after obtaining the VMC estimate.
- The average cost metric ($SE^2 \times \text{Time}$) follows the trend $CV < AV < VMC$ for most values of N in figure 6, except at $N = 10$, where the metric is higher for CV than AV due to the additional computational time outweighing the reduction in variance. However, for larger N , both AV and CV consistently outperform VMC, with CV achieving the lowest overall cost across all N .

Pseudo-Random Numbers

Pseudo-Random number generators like those in Numpy generate random numbers which approximate properties of sequence of random numbers but these are not actually random. Numbers are generated in accordance using a definite function such that

$$x_1 = f(x_0), x_2 = f(f(x_0)), \dots, x_n = f^n(x_0) \text{ where } x_0 \text{ is the seed}$$

The random variables Z and U used in VMC, AV, and CV were generated using pseudo-random number generators. These generators make sequences deterministic in nature, meaning each value is derived from a previous seed value, which can introduce subtle dependencies between successive numbers. As a result, the generated samples may exhibit weak correlations rather than being perfectly independent, which slightly deviates from the theoretical assumption of independent and identically distributed random variables used in Monte Carlo methods.

Sobol Sequence Method

The Sobol method is a quasi-Monte Carlo technique that replaces purely random sampling with low-discrepancy sequences designed to cover the simulation space more uniformly. Unlike standard Monte Carlo paths generated from pseudo-random numbers, Sobol sequences reduce clustering and gaps between sampled points, ensuring a more structured and evenly distributed exploration of the input space. In financial simulations, this uniformity plays a key role in improving efficiency. Traditional pseudo-random generators can still produce uneven coverage, where some regions of the distribution are oversampled while others are underrepresented, leading to higher variance and slower convergence in pricing estimates. Sobol sequences mitigate this issue by systematically filling the space, which results in smoother and more stable estimates of expected payoff.

This more structured sampling significantly improves convergence speed and accuracy, especially in high-dimensional simulations commonly encountered in basket-based and path-dependent structured products like ACBRCs. Since these products depend on multiple underlying assets and their joint evolution over time, maintaining uniform coverage across all dimensions is crucial for capturing rare events such as barrier breaches and autocall triggers more reliably.

Overall, the Sobol method enhances the Monte Carlo framework by reducing simulation noise, improving convergence, and delivering more efficient and stable pricing estimates compared to standard pseudo-random approaches.

Unlike Vanilla Monte Carlo, the standard formula for estimating standard error (SE) based on sample variance is not directly applicable in the same way for Sobol-based simulations because the samples are not truly independent and identically distributed (IID). This breaks the classical statistical assumptions used in VMC for SE estimation. In VMC, the standard error is typically computed as

$$SE_{VMC} = \frac{\sqrt{\text{Var}(\hat{Y})}}{\sqrt{N}} = \frac{s}{\sqrt{N}},$$

where s^2 is the sample variance and N is the number of simulated paths. However, for Sobol sequences, this direct interpretation is not valid due to deterministic low-discrepancy sampling. As a result, error estimation in Sobol methods is often more heuristic or based on replication strategies.

To address this, a common approach is random shifting or budgeting, where multiple independent Sobol runs (each with a different random shift or scrambling) are performed. Suppose we split the total computational budget into M independent Sobol replications, each producing an estimate $\hat{Y}_i$. Then the overall estimator is

$$\hat{Y} = \frac{1}{M} \sum_{i=1}^M \hat{Y}_i,$$

and the standard error is computed across replications as

$$SE_{Sobol} = \frac{\sqrt{\text{Var}(\hat{Y}_i)}}{\sqrt{M}}.$$

The total computational budget is matched with Vanilla Monte Carlo by ensuring that the total number of simulated paths $N_{total} = M \times N$ remains comparable, even though Sobol paths are structured rather than purely random.

Overall, Sobol sequences improve convergence speed and reduce simulation noise, while budgeting techniques allow fair comparison with Vanilla Monte Carlo by restoring a meaningful estimate of variability and standard error under a quasi-Monte Carlo framework.

A key challenge in applying Sobol sequences to high-dimensional financial simulations is that the low-discrepancy properties of Sobol sequences are strongest in the first few dimensions and deteriorate as dimensionality increases. To address this, two dimensionality reduction techniques were applied to ensure that the most important sources of variance are captured in the leading Sobol dimensions.

Principal Component Analysis (PCA)

PCA decomposes the covariance matrix Σ of the asset returns into its eigenvalues and eigenvectors:

$$\Sigma = V\Lambda V^T$$

where V is the matrix of eigenvectors and Λ is the diagonal matrix of eigenvalues sorted in descending order. The correlated random variables are then constructed as:

$$Z_{corr} = V\Lambda^{0.5} \cdot Z$$

By ordering the principal components from largest to smallest variance, the most significant dimensions of the simulation are assigned to the leading — and most uniform — dimensions of the Sobol sequence. This ensures that the variance reduction benefits of Sobol are concentrated where they matter most.

Cholesky-PCA

Cholesky-PCA combines the correlation structure from Cholesky decomposition with the variance ordering of PCA. The correlation matrix is first decomposed using Cholesky factorization:

$$\Sigma = LL^T$$

The resulting lower triangular matrix L is then reordered using PCA so that dimensions contributing the most variance appear first. This hybrid approach retains the exact correlation structure whilst simultaneously aligning the highest-variance dimensions with the leading Sobol dimensions, offering a more structured and efficient exploration of the simulation space compared to either method alone.

Both techniques improve the effectiveness of Sobol sampling by ensuring that low-discrepancy coverage is applied where it has the greatest impact on pricing accuracy, particularly for basket-based path-dependent products such as ACBRCs where multi-asset correlation plays a critical role in determining the final payoff.

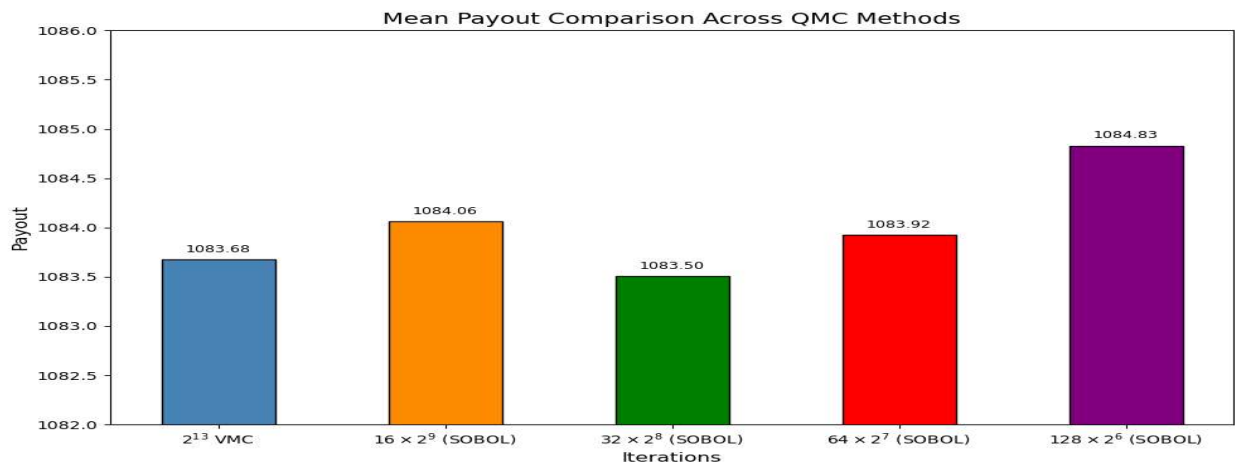
RESULTS:

To determine and compare the performances of VMC and Sobol based QMC approaches following experiments were carried out:

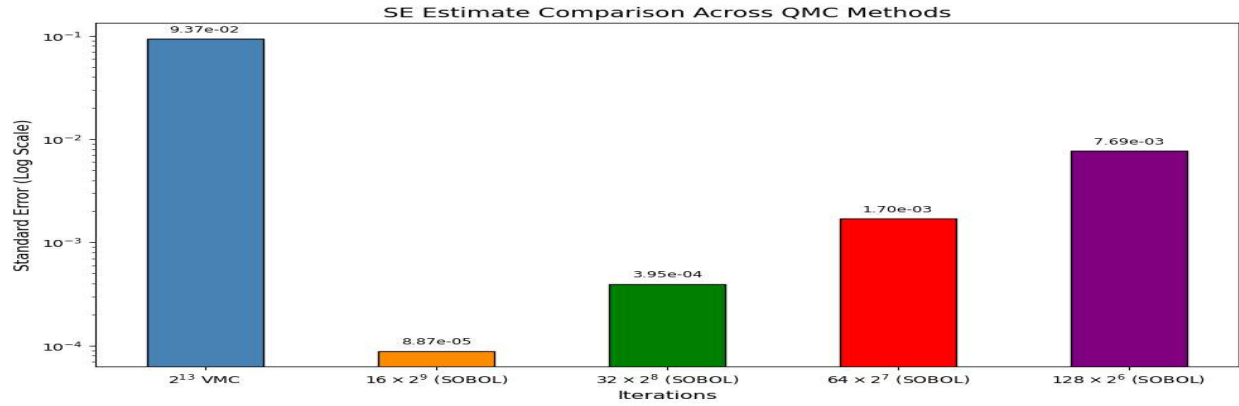
VMC with 50 iterations and 2^{13} simulations were compared to 50 iterations of 2^9 simulations * 2^4 iterations (i.e. 16 iterations), 2^8 simulations * 2^5 iterations (i.e. 32 iterations), 2^7 simulations * 2^6 iterations (i.e. 64 iterations) and 2^6 simulations * 2^7 iterations (i.e. 128 iterations)

FINDINGS:

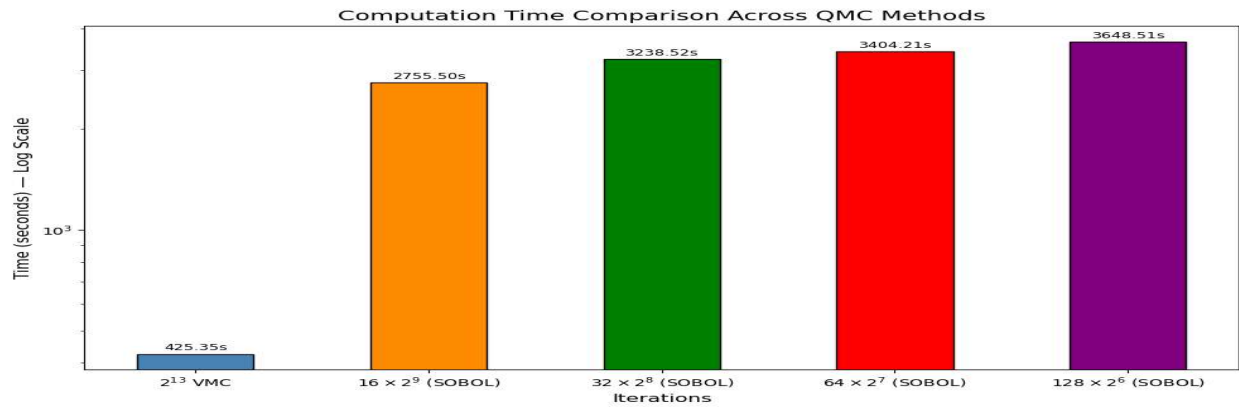
- Sobol techniques for all combinations of iterations and simulation produce a similar mean payout to VMC ranging from 1083.5 to 1084.43 (in figure 7)
- The Sobol, with 16 iterations and 2^9 simulations produce the best SE which is more than 100x better than VMC (in figure 8)
- Mean payout for 128×2^6 produces relatively higher SE among all the Sobol and the difference between the mean payout is also large. This combination is not the best.
- The time taken by Sobol is upwards of 50 minutes which is considerably large when compared to VMC taking <8 mins for the computation. Sobol sequences require structured deterministic construction and scrambling to ensure low-discrepancy coverage, making each sample computationally more expensive than the simple random draws used in VMC. (in figure 9)
- Although Sobol sequences require significantly greater computational time than VMC, their cost function ($SE^2 \times Time$) is exponentially smaller due to the dramatically reduced standard error, demonstrating that Sobol methods are far more efficient in terms of the precision-to-cost tradeoff. (in figure 10)



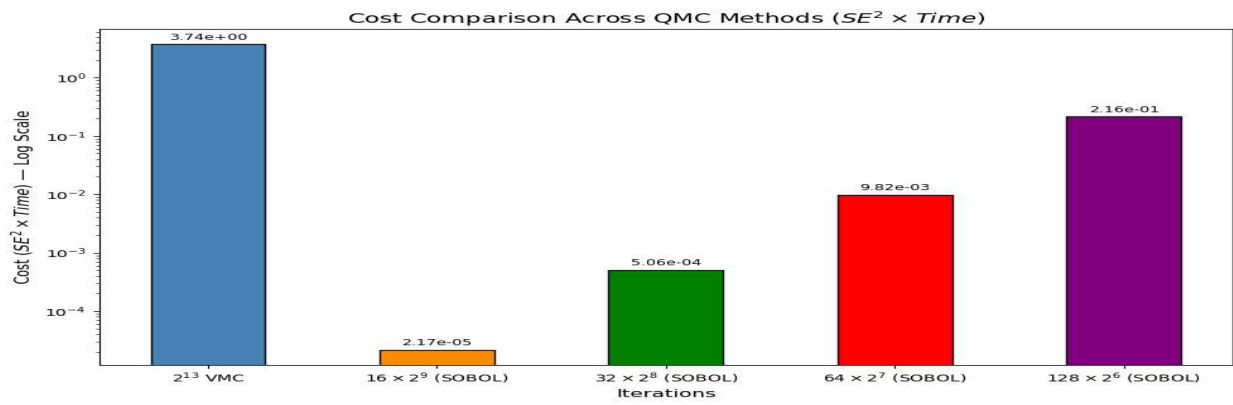
(Figure 7)



(Figure 8)



(Figure 9)



(Figure 10)

Conclusions

This project investigated the pricing of Auto Call Barrier Reverse Convertibles (ACBRCs) using Monte Carlo simulation methods, comparing the performance of Vanilla Monte Carlo (VMC), Antithetic Variates (AV), Control Variates (CV), and Sobol Quasi-Monte Carlo approaches across key metrics including mean payout, standard error, computation time, and cost efficiency.

The results demonstrate that all techniques converge to a consistent mean payout of approximately 1083–1085 SGD, confirming that each method produces unbiased estimates of the true product price. However, significant differences emerge in terms of variance reduction and computational efficiency.

Among the classical variance reduction techniques, Control Variates achieved the most substantial improvement, reducing standard error by 30–40% relative to VMC, while Antithetic Variates delivered a more moderate reduction of 15–20% with comparatively lower computational overhead. The cost metric $SE^2 \times \text{Time}$ confirmed that CV consistently outperforms both VMC and AV for larger simulation sizes, establishing it as the preferred classical technique for pricing path-dependent structured products such as ACBRCs. Antithetic Variates, while less effective in variance reduction, offers a practical advantage in computational efficiency as the number of simulations increases, making it a viable alternative when speed is prioritized over precision.

The Sobol Quasi-Monte Carlo approach demonstrated the most significant improvement in standard error across all methods evaluated. The optimal configuration of 16×2^9 iterations achieved a standard error reduction of over 1000x compared to VMC, representing a substantial gain in simulation precision. Although Sobol sequences require considerably greater computation time due to the structured deterministic construction and scrambling process involved, the cost function $SE^2 \times \text{Time}$ remains exponentially smaller than that of VMC, confirming that Sobol methods are far superior in terms of precision-to-cost efficiency when the full computational budget is considered.

In conclusion, for pricing complex path-dependent structured products such as ACBRCs, Control Variates represents the most effective classical approach, offering meaningful variance reduction with manageable computational cost. For applications where maximum precision is required and computational resources permit, Sobol Quasi-Monte Carlo with 16×2^9 iterations is the recommended approach, delivering unmatched efficiency in terms of the precision-to-cost tradeoff across all methods investigated in this project.

References

Hull, J.C. (2017). *Options, Futures, and Other Derivatives*. 10th edn. New Jersey: Pearson.

Natenberg, S. (1994). *Option Volatility and Pricing: Advanced Trading Strategies and Techniques*. 2nd edn. New York: McGraw-Hill.

Glasserman, P. (2003). *Monte Carlo Methods in Financial Engineering*. New York: Springer.